# Day 16 Task Workflow — Wednesday, Mar 26, 2026

> **Sprint:** 5 (Feature Depth) | **Phase:** PCAP Analysis Pipeline + CICIDS2017 Validation + ml_models Table + Admin Module
> **Owner:** Lead Architect | **Status:** ⚪ Ready to Start
> **Goal:** Complete remaining v0.5.0 backend tasks: PCAP processor, CICIDS2017 cross-dataset validation, populate ml_models table, admin endpoints
> **Grade:** Week 4 Day 2 COMPLETE ☑ | Week 5 Day 1 STARTING ⚫

## Day 15 Results Context

100% VPS verification pass rate on all 6 tasks:

```
Day 15 Achievements:
  ☑ Reports Module: 3/3 endpoints (generate, list, download) — JSON format
  ☑ System Config: GET /system/config operational — no API keys exposed
  ☑ Alert IOC Enrichment: ioc_enrichment field in GET /alerts/{id}
  ☑ IF Retrain: Model artifact updated (1.47→1.57 MB), tuned params active
  ☑ Alert Cleanup: 24 old TM-ALERT-* test alerts removed, 124 real alerts
preserved
  ☑ CICIDS2017 Loader: Full CICIDS2017Loader class (413 lines) verified on VPS
  ☑ API Coverage: 42/42 (100%) 🎯

v0.5.0 Backend Status:
  PCAP processor: ❌ Missing (upload endpoint has graceful ImportError fallback)
  CICIDS2017 validation: ⚠ Loader ready, validation not run (needs dataset)
  ml_models table: ❌ Empty (file-based models work, table needed for ML Ops
dashboard)
  Admin endpoints: ❌ Not started (Week 6 per timeline, but can scaffold early)

Container Status:
  tm-backend     ☑ Up (rebuilt Day 15)
  tm-capture     ☑ Up 5+ days
  tm-ml-worker   ☑ Up (rebuilt Day 15, tuned IF)
  tm-postgres    ☑ Healthy 5+ days
  tm-redis       ☑ Healthy 5+ days
```

## Scope Adherence Checklist

| Requirement | Source Document | Section | Status |
|---|---|---|---|
| PCAP analysis pipeline (processor) | MASTER_DOC_PART5 | Week 5, Lead task | ❌ **DO TODAY** |
| CICIDS2017 validation | MASTER_DOC_PART4 | §3, PART5 Week 5 | ❌ **DO TODAY** |
| ML model registry population | MASTER_DOC_PART3 | §9.1 (Model Registry) | ❌ **DO TODAY** |

| Requirement | Source Document | Section | Status |
|---|---|---|---|
| Confusion matrix data endpoint | MASTER_DOC_PART3 | §9.1 (Confusion Matrix) | ❌ **DO TODAY** |
| Feature importance endpoint | MASTER_DOC_PART3 | §9.1 (Feature Importance) | ❌ **DO TODAY** |
| Audit log endpoint | MASTER_DOC_PART3 | §11.1 (Audit Log) | ⏳ Optional Day 16 |
| Ensemble weights (0.30/0.45/0.25) | MASTER_DOC_PART4 | §1.2 | 🔒 LOCKED |
| Alert thresholds (0.90/0.75/0.50/0.30) | MASTER_DOC_PART4 | §1.2 | 🔒 LOCKED |

## LOCKED CONSTRAINTS (DO NOT MODIFY)

```
Ensemble Weights:
  composite = 0.30 × IF + 0.45 × RF + 0.25 × AE

Alert Thresholds:
  ≥ 0.90 → CRITICAL
  ≥ 0.75 → HIGH
  ≥ 0.50 → MEDIUM
  ≥ 0.30 → LOW
  < 0.30 → NONE

LLM Provider: OpenRouter only (3 verified models)
DO NOT suggest: Kafka, Kubernetes, Elasticsearch, MongoDB
```

---

## Day 16 Objective

By end of day:

- `pcap_processor.py` implemented — PCAP upload → flow extraction → ML scoring pipeline
- CICIDS2017 validation executed — ensemble evaluated on secondary dataset, results saved
- `ml_models` table populated with current model metadata (3 models)
- ML Ops data endpoints enhanced: confusion matrix + feature importance + training history
- Admin audit log read endpoint scaffolded

> **NOTE:** All manual testing and VPS verification performed by Lead Architect.

---

## Task Breakdown

### TASK 1 — PCAP Processor Implementation 🔴

**Time Est:** 90 min | **Priority:** 🔴 Critical — Completes PCAP analysis pipeline
**Source:** MASTER_DOC_PART5 Week 5 ("PCAP upload + analysis pipeline"), MASTER_DOC_PART3 §8 (Forensics Lab)

The PCAP upload endpoint (`POST /capture/upload-pcap`) exists and accepts file uploads, but `pcap_processor.py` is missing. When a PCAP is uploaded, it currently returns `{"status": "processing"}` but does no actual analysis.

### 1.1 Create `backend/app/services/pcap_processor.py`

The processor should:

1. Read the uploaded PCAP file using Scapy
2. Extract flows (group packets by 5-tuple)
3. Calculate features per flow (reuse capture engine feature extraction)
4. Score each flow with the ML ensemble
5. Store results in `network_flows` table with `source='pcap'`
6. Update the `pcap_uploads` table with results

```
"""
ThreatMatrix AI — PCAP File Processor

Per MASTER_DOC_PART3 §8 (Forensics Lab):
  Upload .pcap/.pcapng → extract flows → ML scoring → store results.

Per MASTER_DOC_PART5 Week 5:
  "Upload and analyze historical traffic"

Pipeline:
  1. Read PCAP with Scapy
  2. Group packets into flows (5-tuple)
  3. Extract features per flow
  4. Score with ML ensemble
  5. Persist to network_flows (source='pcap')
  6. Update pcap_uploads record
"""

from __future__ import annotations

import asyncio
import logging
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Dict, List, Optional
from uuid import uuid4

from sqlalchemy import text

from app.database import async_session

logger = logging.getLogger(__name__)
```

```python
class PcapProcessor:
    """Process uploaded PCAP files for forensic analysis."""

    def __init__(self) -> None:
        self.stats: Dict[str, Any] = {
            "packets_read": 0,
            "flows_extracted": 0,
            "anomalies_found": 0,
        }

    async def process(
        self,
        pcap_path: str,
        upload_id: str,
    ) -> Dict[str, Any]:
        """
        Process a PCAP file end-to-end.

        Args:
            pcap_path: Path to the uploaded PCAP file.
            upload_id: UUID of the pcap_uploads record.

        Returns:
            Processing results dict.
        """
        logger.info("[PCAP] Processing: %s (upload_id=%s)", pcap_path, upload_id)

        try:
            # 1. Read PCAP
            packets = self._read_pcap(pcap_path)
            self.stats["packets_read"] = len(packets)

            # 2. Group into flows
            flows = self._extract_flows(packets)
            self.stats["flows_extracted"] = len(flows)

            # 3. Extract features per flow
            flow_features = [self._extract_features(f) for f in flows]

            # 4. Score with ML ensemble
            scored_flows = await self._score_flows(flow_features)

            # 5. Count anomalies
            anomalies = [f for f in scored_flows if f.get("is_anomaly")]
            self.stats["anomalies_found"] = len(anomalies)

            # 6. Persist to database
            await self._persist_flows(scored_flows, upload_id)
            await self._update_upload_record(upload_id)

            logger.info(
                "[PCAP] Complete: %d packets, %d flows, %d anomalies",
```

```
                self.stats["packets_read"],
                self.stats["flows_extracted"],
                self.stats["anomalies_found"],
            )
            return self.stats

        except Exception as e:
            logger.error("[PCAP] Processing failed: %s", e)
            await self._update_upload_record(upload_id, status="failed")
            raise

    def _read_pcap(self, pcap_path: str) -> list:
        """Read packets from PCAP file using Scapy."""
        try:
            from scapy.all import rdpcap
            packets = rdpcap(pcap_path)
            logger.info("[PCAP] Read %d packets from %s", len(packets), pcap_path)
            return list(packets)
        except ImportError:
            logger.error("[PCAP] Scapy not available")
            return []
        except Exception as e:
            logger.error("[PCAP] Failed to read %s: %s", pcap_path, e)
            return []

    def _extract_flows(self, packets: list) -> List[Dict]:
        """Group packets into flows by 5-tuple."""
        flows: Dict[str, Dict] = {}

        for pkt in packets:
            try:
                if not hasattr(pkt, 'ip'):
                    # Try to get IP layer
                    if hasattr(pkt, 'payload') and hasattr(pkt.payload, 'src'):
                        ip_layer = pkt.payload
                    else:
                        continue
                else:
                    ip_layer = pkt.ip

                src_ip = str(getattr(ip_layer, 'src', '0.0.0.0'))
                dst_ip = str(getattr(ip_layer, 'dst', '0.0.0.0'))
                proto = int(getattr(ip_layer, 'proto', 0))

                src_port = 0
                dst_port = 0
                if hasattr(pkt, 'sport'):
                    src_port = int(pkt.sport)
                    dst_port = int(pkt.dport)
                elif hasattr(ip_layer, 'payload'):
                    tcp_udp = ip_layer.payload
                    src_port = int(getattr(tcp_udp, 'sport', 0))
                    dst_port = int(getattr(tcp_udp, 'dport', 0))
```

```
                key = f"{src_ip}:{src_port}-{dst_ip}:{dst_port}-{proto}"

                if key not in flows:
                    flows[key] = {
                        "src_ip": src_ip,
                        "dst_ip": dst_ip,
                        "src_port": src_port,
                        "dst_port": dst_port,
                        "protocol": proto,
                        "packets": [],
                        "total_bytes": 0,
                        "start_time": float(pkt.time),
                    }

                flows[key]["packets"].append(pkt)
                flows[key]["total_bytes"] += len(pkt)
                flows[key]["end_time"] = float(pkt.time)

            except Exception:
                continue

        logger.info("[PCAP] Extracted %d flows from %d packets", len(flows),
len(packets))
        return list(flows.values())

    def _extract_features(self, flow: Dict) -> Dict[str, Any]:
        """Extract ML features from a flow."""
        n_pkts = len(flow["packets"])
        duration = flow.get("end_time", 0) - flow.get("start_time", 0)
        total_bytes = flow.get("total_bytes", 0)

        return {
            "src_ip": flow["src_ip"],
            "dst_ip": flow["dst_ip"],
            "src_port": flow["src_port"],
            "dst_port": flow["dst_port"],
            "protocol": flow["protocol"],
            "duration": max(duration, 0.001),
            "total_bytes": total_bytes,
            "total_packets": n_pkts,
            "src_bytes": total_bytes // 2,  # Approximation
            "dst_bytes": total_bytes // 2,
            "packets_per_second": n_pkts / max(duration, 0.001),
            "bytes_per_packet": total_bytes / max(n_pkts, 1),
        }

    async def _score_flows(self, flow_features: List[Dict]) -> List[Dict]:
        """Score flows with ML ensemble."""
        if not flow_features:
            return []

        try:
            from ml.inference.preprocessor import FlowPreprocessor
            from ml.inference.model_manager import ModelManager
```

```
            import numpy as np

            preprocessor = FlowPreprocessor()
            preprocessor.load()

            manager = ModelManager()
            manager.load_all()

            for features in flow_features:
                try:
                    X = preprocessor.preprocess_flow(features)
                    if X is not None:
                        results = manager.score_flows(X.reshape(1, -1))
                        if results:
                            r = results[0]
                            features["anomaly_score"] = r["composite_score"]
                            features["is_anomaly"] = r["is_anomaly"]
                            features["label"] = r["label"]
                            features["severity"] = r["severity"]
                    else:
                        features["anomaly_score"] = 0.0
                        features["is_anomaly"] = False
                except Exception:
                    features["anomaly_score"] = 0.0
                    features["is_anomaly"] = False

        except ImportError:
            logger.warning("[PCAP] ML models not available for scoring")
            for f in flow_features:
                f["anomaly_score"] = 0.0
                f["is_anomaly"] = False

        return flow_features

    async def _persist_flows(self, flows: List[Dict], upload_id: str) -> None:
        """Persist scored flows to network_flows table."""
        async with async_session() as session:
            for flow in flows:
                await session.execute(
                    text("""
                        INSERT INTO network_flows (
                            id, timestamp, src_ip, dst_ip, src_port, dst_port,
                            protocol, duration, total_bytes, total_packets,
                            anomaly_score, is_anomaly, label, source, features
                        ) VALUES (
                            :id, :ts, :src_ip, :dst_ip, :src_port, :dst_port,
                            :protocol, :duration, :total_bytes, :total_packets,
                            :anomaly_score, :is_anomaly, :label, 'pcap', :features
                        )
                    """),
                    {
                        "id": str(uuid4()),
                        "ts": datetime.now(timezone.utc),
                        "src_ip": flow["src_ip"],
```

```
                        "dst_ip": flow["dst_ip"],
                        "src_port": flow["src_port"],
                        "dst_port": flow["dst_port"],
                        "protocol": flow["protocol"],
                        "duration": flow.get("duration", 0),
                        "total_bytes": flow.get("total_bytes", 0),
                        "total_packets": flow.get("total_packets", 0),
                        "anomaly_score": flow.get("anomaly_score", 0),
                        "is_anomaly": flow.get("is_anomaly", False),
                        "label": flow.get("label"),
                        "features": "{}",
                    },
                )
            await session.commit()
        logger.info("[PCAP] Persisted %d flows (source=pcap)", len(flows))

    async def _update_upload_record(
        self, upload_id: str, status: str = "completed"
    ) -> None:
        """Update pcap_uploads table with processing results."""
        async with async_session() as session:
            await session.execute(
                text("""
                    UPDATE pcap_uploads
                    SET status = :status,
                        packets_count = :packets,
                        flows_extracted = :flows,
                        anomalies_found = :anomalies,
                        processed_at = :now,
                        updated_at = :now
                    WHERE id::text = :upload_id
                """),
                {
                    "status": status,
                    "packets": self.stats["packets_read"],
                    "flows": self.stats["flows_extracted"],
                    "anomalies": self.stats["anomalies_found"],
                    "now": datetime.now(timezone.utc),
                    "upload_id": upload_id,
                },
            )
            await session.commit()
```

### 1.2 Integrate with capture.py Upload Endpoint

Update `backend/app/api/v1/capture.py` — the `POST /capture/upload-pcap` handler to call the processor:

```
# In the upload-pcap endpoint, after saving the file:
import asyncio
from app.services.pcap_processor import PcapProcessor
```

```
processor = PcapProcessor()
asyncio.create_task(processor.process(file_path, upload_id))
```

**Verification:**

| # | Check | Expected |
|---|---|---|
| 1 | Upload valid .pcap file | 200, task_id returned |
| 2 | `pcap_uploads` table updated | status=completed, packets/flows/anomalies counts |
| 3 | `network_flows` table has source='pcap' rows | New rows with PCAP-derived features |
| 4 | ML scoring applied to PCAP flows | anomaly_score populated |
| 5 | Invalid file type rejected | 400, .pcap/.pcapng/.cap only |
| 6 | Error handling | Failed processing → status=failed, no crash |

---

## TASK 2 — CICIDS2017 Validation Run 🟤

**Time Est:** 45 min | **Priority:** 🟤 Critical — Academic credibility
**Source:** MASTER_DOC_PART4 §3, MASTER_DOC_PART5 Week 5

The `CICIDS2017Loader` class (413 lines) exists. Now execute the validation.

### 2.1 Download CICIDS2017 Dataset (Subset)

The full dataset is ~6.5 GB. For validation, download one day file:

```
# On VPS — create directory and download a subset:
mkdir -p /app/ml/saved_models/datasets/cicids2017/

# Download Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv (~170 MB)
# This contains BENIGN + DDoS labels — good for binary validation
wget -O /app/ml/saved_models/datasets/cicids2017/Friday-DDos.csv \
  "https://www.kaggle.com/datasets/cicdataset/cicids2017/download"
# Alternative: Use a pre-hosted mirror or kaggle CLI
```

> **NOTE:** If the dataset cannot be downloaded on VPS (network/storage constraints), create a synthetic validation dataset mapping CICIDS2017-style features for testing.

### 2.2 Execute Validation

```
docker compose exec backend python -c "
from ml.datasets.cicids2017 import validate_ensemble_on_cicids2017
results = validate_ensemble_on_cicids2017()
print(f'Accuracy: {results.get(\"ensemble\", {}).get(\"accuracy\", \"N/A\")}')
```

```
print(f'F1: {results.get(\"ensemble\", {}).get(\"f1_score\", \"N/A\")}')
"
```

#### 2.3 Save + Expose Results

Results should be saved to `ml/saved_models/eval_results/cicids2017_validation.json` and accessible via `GET /ml/comparison`.

**Verification:**

| # | Check | Expected |
|---|---|---|
| 1 | `validate_ensemble_on_cicids2017()` runs without error | Returns results dict |
| 2 | Results JSON saved | `cicids2017_validation.json` exists |
| 3 | Accuracy reported | Numeric value (any range acceptable for cross-dataset) |
| 4 | F1 reported | Numeric value |
| 5 | `GET /ml/comparison` includes CICIDS2017 results | Listed in model comparison |
| 6 | Label distribution logged | Class breakdown visible |

---

### TASK 3 — Populate ml_models Table 🟤

**Time Est:** 30 min | **Priority:** 🟤 Critical — Enables ML Ops dashboard
**Source:** MASTER_DOC_PART3 §9.1 (Model Registry)

The `ml_models` table exists (schema defined in Day 6) but has 0 rows. The ML Ops dashboard (frontend) needs this populated.

#### 3.1 Create Model Registry Population Script

```
"""Populate ml_models table with current model metadata."""

import json
import os
from datetime import datetime, timezone
from pathlib import Path
from uuid import uuid4

from sqlalchemy import text
from app.database import async_session

MODELS_DIR = Path("/app/ml/saved_models")
EVAL_DIR = MODELS_DIR / "eval_results"

async def populate_ml_models():
```

```
    """Insert or update model registry entries."""
    models = [
        {
            "name": "isolation_forest_v1",
            "model_type": "isolation_forest",
            "version": "1.1",  # v1.1 = tuned params
            "status": "active",
            "dataset": "nsl_kdd",
            "file_path": "ml/saved_models/isolation_forest.pkl",
            "eval_file": "isolation_forest_eval.json",
        },
        {
            "name": "random_forest_v1",
            "model_type": "random_forest",
            "version": "1.0",
            "status": "active",
            "dataset": "nsl_kdd",
            "file_path": "ml/saved_models/random_forest.pkl",
            "eval_file": "random_forest_eval.json",
        },
        {
            "name": "autoencoder_v1",
            "model_type": "autoencoder",
            "version": "1.0",
            "status": "active",
            "dataset": "nsl_kdd",
            "file_path": "ml/saved_models/autoencoder/model.keras",
            "eval_file": "autoencoder_eval.json",
        },
    ]

    async with async_session() as session:
        for m in models:
            # Load eval results if available
            metrics = {}
            eval_path = EVAL_DIR / m["eval_file"]
            if eval_path.exists():
                with open(eval_path) as f:
                    metrics = json.load(f)

            # Load hyperparams
            hyperparams = {}
            best_params_path = MODELS_DIR / "best_params.json"
            if best_params_path.exists():
                with open(best_params_path) as f:
                    all_params = json.load(f)
                    hyperparams = all_params.get(m["model_type"], {})

            # Get model file stats
            model_path = Path("/app") / m["file_path"]
            file_size = model_path.stat().st_size if model_path.exists() else 0

            await session.execute(
                text("""
```

```
                    INSERT INTO ml_models (
                        id, name, model_type, version, status, dataset,
                        metrics, hyperparams, file_path, is_active,
                        trained_at, created_at, updated_at
                    ) VALUES (
                        :id, :name, :model_type, :version, :status, :dataset,
                        :metrics, :hyperparams, :file_path, true,
                        :trained_at, :now, :now
                    )
                    ON CONFLICT (id) DO NOTHING
                """),
                {
                    "id": str(uuid4()),
                    "name": m["name"],
                    "model_type": m["model_type"],
                    "version": m["version"],
                    "status": m["status"],
                    "dataset": m["dataset"],
                    "metrics": json.dumps(metrics),
                    "hyperparams": json.dumps(hyperparams),
                    "file_path": m["file_path"],
                    "trained_at": datetime.now(timezone.utc),
                    "now": datetime.now(timezone.utc),
                },
            )
        await session.commit()
```

### 3.2 Alternative: Add Startup Population

Add to `backend/app/main.py` startup to auto-populate:

```
@app.on_event("startup")
async def populate_models_registry():
    """Ensure ml_models table has current model entries."""
    # ... call populate function
```

**Verification:**

| # | Check | Expected |
|---|---|---|
| 1 | `SELECT COUNT(*) FROM ml_models` | 3 (IF, RF, AE) |
| 2 | `SELECT name, model_type, status, is_active FROM ml_models` | All 3 active |
| 3 | Metrics JSONB populated | accuracy, f1, etc. from eval results |
| 4 | Hyperparams populated | IF tuned params visible |
| 5 | `GET /ml/models` includes registry data | Models list with DB metadata |

| # | Check | Expected |
|---|---|---|
| 6 | File paths correct | Point to actual model files |

---

## TASK 4 — ML Ops Data Endpoints (Confusion Matrix + Feature Importance) ⚪

**Time Est:** 60 min | **Priority:** ⚪ Medium
**Source:** MASTER_DOC_PART3 §9.1 (ML Ops Module)

The ML Ops module spec requires confusion matrix and feature importance data. These need dedicated endpoints.

### 4.1 Add to `backend/app/api/v1/ml.py`

```
@router.get("/models/{model_type}/confusion-matrix")
async def get_confusion_matrix(model_type: str) -> Dict[str, Any]:
    """
    Get confusion matrix for a specific model.
    Per MASTER_DOC_PART3 §9.1 (Confusion Matrix component).
    """
    valid = {"isolation_forest", "random_forest", "autoencoder", "ensemble"}
    if model_type not in valid:
        raise HTTPException(400, f"Invalid model. Valid: {valid}")

    eval_path = EVAL_DIR / f"{model_type}_eval.json"
    if not eval_path.exists():
        raise HTTPException(404, f"No evaluation results for {model_type}")

    with open(eval_path) as f:
        eval_data = json.load(f)

    return {
        "model": model_type,
        "confusion_matrix": eval_data.get("confusion_matrix", []),
        "class_names": eval_data.get("class_names",
            eval_data.get("classification_report", {}).keys() if
"classification_report" in eval_data else ["normal", "anomaly"]),
        "n_samples": eval_data.get("n_samples", 0),
    }


@router.get("/models/{model_type}/feature-importance")
async def get_feature_importance(model_type: str) -> Dict[str, Any]:
    """
    Get feature importance for a specific model.
    Per MASTER_DOC_PART3 §9.1 (Feature Importance component).

    Only available for tree-based models (Random Forest, Isolation Forest).
    """
    if model_type not in {"isolation_forest", "random_forest"}:
        raise HTTPException(400, "Feature importance only for tree-based models")
```

```
    try:
        import joblib
        model_path = MODELS_DIR / f"{model_type}.pkl"
        if not model_path.exists():
            raise HTTPException(404, f"Model file not found: {model_type}")

        model = joblib.load(model_path)
        importances = model.feature_importances_ if hasattr(model,
'feature_importances_') else None

        if importances is None:
            return {"model": model_type, "feature_importance": [], "note": "Model
lacks feature_importances_ attribute"}

        # Load feature names from preprocessor
        feature_names = [f"feature_{i}" for i in range(len(importances))]
        try:
            preprocessor_path = MODELS_DIR / "preprocessor_scaler.pkl"
            if preprocessor_path.exists():
                scaler = joblib.load(preprocessor_path)
                if hasattr(scaler, 'feature_names_in_'):
                    feature_names = list(scaler.feature_names_in_)
        except Exception:
            pass

        # Sort by importance
        indices = importances.argsort()[::-1]
        ranked = [
            {"feature": feature_names[i] if i < len(feature_names) else
f"feature_{i}",
             "importance": float(importances[i]),
             "rank": rank + 1}
            for rank, i in enumerate(indices[:30])  # Top 30
        ]

        return {
            "model": model_type,
            "feature_importance": ranked,
            "n_features": len(importances),
        }

    except ImportError:
        raise HTTPException(500, "joblib not available")


@router.get("/training-history")
async def get_training_history() -> Dict[str, Any]:
    """
    Get training history from ml_models registry.
    Per MASTER_DOC_PART3 §9.1 (Training History component).
    """
    from app.database import async_session
```

```
    async with async_session() as session:
        result = await session.execute(
            text("""
                SELECT name, model_type, version, status, dataset,
                       metrics, hyperparams, training_time, inference_time,
                       is_active, trained_at, created_at
                FROM ml_models
                ORDER BY created_at DESC
                LIMIT 50
            """)
        )
        rows = result.fetchall()

    history = []
    for r in rows:
        metrics = r[5] if isinstance(r[5], dict) else json.loads(r[5]) if r[5]
else {}
        hyperparams = r[6] if isinstance(r[6], dict) else json.loads(r[6]) if r[6]
else {}
        history.append({
            "name": r[0],
            "model_type": r[1],
            "version": r[2],
            "status": r[3],
            "dataset": r[4],
            "metrics": metrics,
            "hyperparams": hyperparams,
            "training_time": r[7],
            "inference_time": r[8],
            "is_active": r[9],
            "trained_at": str(r[10]) if r[10] else None,
            "created_at": str(r[11]) if r[11] else None,
        })

    return {"history": history, "total": len(history)}
```

**Verification:**

| # | Check | Expected |
|---|---|---|
| 1 | `GET /ml/models/ensemble/confusion-matrix` | 200, matrix array + class names |
| 2 | `GET /ml/models/random_forest/confusion-matrix` | 200, matrix array |
| 3 | `GET /ml/models/invalid/confusion-matrix` | 400 |
| 4 | `GET /ml/models/random_forest/feature-importance` | 200, ranked features list |
| 5 | `GET /ml/models/autoencoder/feature-importance` | 400 (not tree-based) |
| 6 | `GET /ml/training-history` | List of model entries from ml_models table |

| # | Check | Expected |
|---|---|---|
| 7 | Feature importance sorted descending | Rank 1 = most important |
| 8 | Top 30 features returned | Array length ≤ 30 |

---

## TASK 5 — Audit Log Read Endpoint ⚫

**Time Est:** 30 min | **Priority:** ⚫ Low — Scaffolding for Admin module
**Source:** MASTER_DOC_PART3 §11.1 (Administration Module — Audit Log)

The `audit_log` table exists (created in initial migration). Add a read endpoint.

### 5.1 Create `backend/app/api/v1/admin.py`

```
"""
ThreatMatrix AI — Admin API Endpoints

Per MASTER_DOC_PART3 §11 (Administration Module):
  - Audit Log: GET /admin/audit-log
"""

from __future__ import annotations

import json
import logging
from typing import Any, Dict

from fastapi import APIRouter, Query
from sqlalchemy import text

from app.database import async_session

logger = logging.getLogger(__name__)

router = APIRouter(prefix="/admin", tags=["Administration"])


@router.get("/audit-log")
async def get_audit_log(
    limit: int = Query(50, ge=1, le=200),
    offset: int = Query(0, ge=0),
    action: str = Query(None),
) -> Dict[str, Any]:
    """
    Get system audit log.
    Per MASTER_DOC_PART3 §11.1.
    """
    async with async_session() as session:
        query = """
            SELECT id, user_id, action, entity_type, entity_id,
```

```
                    details, ip_address, created_at
            FROM audit_log
        """
        params: Dict[str, Any] = {"limit": limit, "offset": offset}

        if action:
            query += " WHERE action = :action"
            params["action"] = action

        query += " ORDER BY created_at DESC LIMIT :limit OFFSET :offset"

        result = await session.execute(text(query), params)
        rows = result.fetchall()

        count_result = await session.execute(
            text("SELECT COUNT(*) FROM audit_log")
        )
        total = count_result.scalar()

    entries = []
    for r in rows:
        entries.append({
            "id": str(r[0]),
            "user_id": str(r[1]) if r[1] else None,
            "action": r[2],
            "entity_type": r[3],
            "entity_id": str(r[4]) if r[4] else None,
            "details": r[5],
            "ip_address": str(r[6]) if r[6] else None,
            "created_at": str(r[7]),
        })

    return {"entries": entries, "total": total, "limit": limit, "offset": offset}
```

Register in `main.py`:

```
from app.api.v1.admin import router as admin_router
app.include_router(admin_router, prefix="/api/v1")
```

**Verification:**

| # | Check | Expected |
|---|---|---|
| 1 | `GET /admin/audit-log` | 200, entries list (may be empty) |
| 2 | `GET /admin/audit-log?limit=10` | Max 10 entries |
| 3 | `GET /admin/audit-log?action=login` | Filtered by action |
| 4 | OpenAPI docs show new endpoint | Under "Administration" tag |

---

## Files Modified / Created (Expected)

| File | Action | Lines (est.) |
|---|---|---|
| `app/services/pcap_processor.py` | CREATE | ~250 |
| `app/api/v1/capture.py` | MODIFY | +5 (integrate processor) |
| `app/api/v1/ml.py` | MODIFY | +90 (confusion matrix, feature importance, training history) |
| `app/api/v1/admin.py` | CREATE | ~70 |
| `app/main.py` | MODIFY | +5 (admin router, model registry) |
| `scripts/populate_ml_models.py` | CREATE | ~80 |

## Risk Assessment

| Risk | Impact | Mitigation |
|---|---|---|
| Scapy not available in backend container | PCAP parsing fails | Graceful ImportError, ensure `scapy` in requirements.txt |
| CICIDS2017 dataset too large for VPS storage | Cannot run validation | Use single-day CSV subset (~170 MB) or synthetic subset |
| Feature importance not available for IF | Missing `feature_importances_` | Return empty array with note |
| ml_models table has no unique constraint on name | Duplicate entries | Use ON CONFLICT DO NOTHING or check before insert |

## STRICT RULES REMINDER

1. **DO NOT** change ensemble weights (0.30/0.45/0.25) — LOCKED
2. **DO NOT** change alert thresholds (0.90/0.75/0.50/0.30) — LOCKED
3. **DO NOT** suggest Kafka, Kubernetes, Elasticsearch, MongoDB
4. **DO NOT** add features not in the 10 modules
5. **DO NOT** use Tailwind CSS — Vanilla CSS + CSS Variables only
6. LLM via **OpenRouter only** — 3 verified models
7. Master documentation (5 parts) is the **SOLE source of truth**
8. All code: **typed, error-handled, documented, production-quality**
9. Python: **type hints, async/await, SQLAlchemy 2.x**

---

*Day 16 Worklog — Week 5 Day 1*
*v0.4.0 Critical MVP: ACHIEVED ☑ | v0.5.0 Feature Depth: IN PROGRESS*
*API Coverage: 42/42 (100%) 🎯*
*Target: PCAP processor + CICIDS2017 validation + ml_models registry + ML Ops endpoints + admin scaffold*

*Ensemble: 80.73% acc | 80.96% F1 | 0.9312 AUC-ROC (LOCKED)*
*IOC Database: 1,367 indicators (§11.3 fully compliant)*